

Data Understanding

Im Rahmen des CRISP-DM-Modells dient die Phase des Data Understanding der initialen Analyse und Bewertung der zugrundeliegenden Datenbasis, um deren Struktur, Qualität und Eignung für den definierten Anwendungsfall zu verstehen. Im Kontext dieser Arbeit handelt es sich bei den "Daten" nicht um tabellarische Datensätze, sondern um komplexe Software-Artefakte in Form von fünf GitHub-Repositories, die jeweils eine CAP-Anwendung repräsentieren. Die relevanten Merkmale für die Analyse durch den KI-Agenten sind dabei die Verzeichnisstruktur, Inhalte von Konfigurationsdateien wie *package.json* und der *mta.yaml*, Build-Skripte sowie versionsspezifische Unterschiede der Projekte.

Als Datenbasis wurden fünf CAP-Anwendungen ausgewählt. Vier davon - **CAP-SFlight**, **CAP-Orders**, **CAP-Reviews** sowie **CAP-Bookshop** - sind öffentlich verfügbare Schulungsanwendungen von SAP. Diese wurden aufgrund ihrer einfachen und übersichtlichen Struktur, des erleichterten Zugriffs sowie der öffentlichen Validierung ihrer Funktionsfähigkeit als primäre Testfälle für den KI-Agenten gewählt.

Die fünfte Anwendung ist eine interne **Retrieval Augmented Generation (RAG)**-Anwendung, die eine vergleichbar aufgebaute, jedoch deutlich komplexere und umfangreichere Struktur aufweist. Sie dient als finaler Testfall, um die Leistungsfähigkeit des KI-Agenten auch in komplexeren CAP-Szenarien zu evaluieren. Alle für diese Ausarbeitung genutzten CAP-Anwendungen sind prinzipiell voll funktionsfähig und für ein Deployment auf der Cloud Foundry Runtime der SAP BTP ausgelegt.

Explorative Datenanalyse

Im Rahmen der explorativen Analyse wurden die fünf CAP-Anwendungen systematisch untersucht, um Gemeinsamkeiten, Muster und relevante Unterschiede für die automatisierte CI/CD-Pipeline-Generierung zu identifizieren.

Die Analyse bestätigte, dass alle Anwendungen auf dem Cloud Application Programming Model basieren und Node.js als Laufzeitumgebung nutzen. Es konnte eine durchgängige Verwendung von SAP HANA Cloud als Datenbank sowie XSUAA für die Authentifizierung festgestellt werden. Die Projekte folgen der typischen CAP-Struktur mit den Kernverzeichnissen *db/*, *srv/* und *app/* und verwenden standardisierte Build- und Deploy-Befehle. Diese technologische Homogenität schafft eine valide Grundlage für die automatisierte Analyse durch den Agenten.

Gleichzeitig traten signifikante Unterschiede zutage, die für die Konzeption des Agenten von entscheidender Bedeutung sind. Wie in der nachfolgenden Tabelle ersichtlich, variieren die eingesetzten CAP- und Node.js-Versionen, was zu Unterschieden in der Command Line Interface (CLI) und den Build-Prozessen führen kann. Zudem weisen die Projekte eine unterschiedliche Komplexität auf, die sich in der Anzahl der Abhängigkeiten, dem Vorhandensein von UI-Komponenten oder der Integration zusätzlicher Services (z. B. AI SDKs) manifestiert.

Merkm ^{al}	CAP-Bookshop	CAP-SFlight	CAP-Orders	CAP-Reviews	CAP-RAG
CAP Version	v6.3.0	v5.9.8	v7.1.0	v7.1.0	v8.0.1
Node.js Version	16.x	14.x	18.x	18.x	18.x
DB-Technologie	HANA Cloud	SQLite	HANA Cloud	HANA Cloud	HANA Cloud
<i>mta.yaml</i> vorhanden?	Nein	Nein	Nein	Ja	Ja
UI-Komponente	Fiori UI	Keine	Keine	Fiori UI	UI5 Freestyle

Zusätzl. Services	Keine	Keine	Keine	Keine	AI SDK, XSUAA
Anzahl Dependencies	25	18	22	23	42

Datenqualität

Die Untersuchung der Datenqualität offenbarte mehrere Mängel, die ein direktes Deployment der Anwendungen verhindern.

Vollständigkeit: Bei drei der fünf Repositories fehlte die *mta.yaml*-Datei. Diese ist jedoch essenziell für das Deployment auf die Cloud Foundry Runtime, da sie die gesamte Anwendungsarchitektur als Multi-Target Application (MTA) definiert.

Korrektheit: In einigen Projekten waren Build-Skripte in der *package.json*-Datei veraltet oder fehlerhaft, was zu Fehlern im Build-Prozess geführt hätte.

Konsistenz: Es wurden unterschiedliche Node.js-Versionen und CAP-Versionen identifiziert, was eine flexible Pipeline-Logik erfordert, die diese Unterschiede berücksichtigen kann.

Die Behebung und Aufarbeitung der dieser Mängel wird im Kapitel Data Preparation (5) detailliert behandelt.

Zusammenfassend lässt sich festhalten, dass die analysierten Projekte zwar eine einheitliche technologische Grundlage bieten, sich jedoch in Komplexität, Vollständigkeit und Konfigurationsstand erheblich unterscheiden. Das daraus gewonnene Verständnis bildet die entscheidende Basis für das nächste Kapitel, die Datenvorbereitung, sowie für die Entwicklung des KI-Agenten. Insbesondere muss der Agent in der Lage sein, projektspezifische Konfigurationen zu interpretieren und seine Prozesse entsprechend anzupassen.